Opgavebeskrivelse: Konvertering af Monolit til Microservices vha Strangler Pattern

Baggrund:

I dette projekt skal I arbejde med omdannelsen af et stort, monolitisk system til en moderne microservice-arkitektur. I vil tage udgangspunkt i projektet **eShopOnWeb**, som er en monolitisk applikation, der simulerer en e-handelsplatform. Jeres opgave bliver gradvist at erstatte dele af systemet ved hjælp af et **Strangler Pattern**, hvilket betyder, at I flytter logikken ud i uafhængige microservices, samtidig med at det eksisterende system forbliver operationelt under overgangen.

Hovedmål:

Målet med projektet er at omdanne eShopOnWeb fra en monolit til en microservice-arkitektur, hvor hver service har klare ansvar, kan skaleres uafhængigt, og implementeres med passende kommunikationsmekanismer mellem tjenesterne. I skal derudover integrere et nyt lagerstyringssystem I har lavet, men ikke er implementeret i eShopOnWeb.

Kravspecifikation:

I skal sikre jer, at følgende krav opfyldes i jeres projekt:

1. **Identificér og afgræns services:** Analysér det eksisterende monolitiske system og identificér de forretningsdomæner, der kan adskilles som uafhængige microservices. I skal opdele monolitten i mindst **3 microservices**, f.eks. til **produktkatalog**, **ordrehåndtering** og **betalingsservice**.
2. **Strangler Pattern implementering:** Brug strangler-patternet til gradvist at overføre funktionalitet fra det monolitiske system til de nye microservices uden at afbryde driften. De dele, der konverteres, skal rutes til microservices, mens de øvrige funktioner stadig kører i monolitten.
3. **Lagerstyringssystem:** Implementér det forberedte lagerstyringssystem i projektet. Systemet skal være ansvarligt for at holde styr på produktbeholdning, modtage opdateringer fra ordreafdelingen og f.eks håndtere reservation af produkter ved checkout. Dette skal implementeres som en selvstændig microservice.
4. **API Gateway:** Implementér en **API Gateway**, der fungerer som et samlet indgangspunkt for klientanmodninger og dirigerer dem til de rette microservices. Den skal også håndtere autentificering, load balancing og rate-limiting.
5. **Kommunikation mellem microservices:** Anvend **asynkron beskedbaseret kommunikation** (f.eks. med RabbitMQ, Azure Service Bus eller Kafka) mellem

microservices til at sikre løs kobling og skalerbarhed, især når det gælder lagerstyring og ordrebehandling.

6. **Datahåndtering og dataseparation:** Hver microservice skal have sin egen database, og I skal sikre, at de services, der har brug for data fra andre services, får det via beskeder eller API-opkald.
7. **Fejlhåndtering og overvågning:** Implementér robuste mekanismer til fejlhåndtering og overvågning af microservices. Dette kan omfatte brug af **Circuit Breaker** mønstre, logging, samt centraliseret overvågning via f.eks. **ELK stack** (Elasticsearch, Logstash, Kibana) eller **Prometheus/Grafana**.
8. **CI/CD pipeline:** Opsæt en **CI/CD pipeline**, som understøtter automatisk bygning, test og deployment af microservices. Pipelinens opgaver inkluderer at køre automatiske tests, containerisering af services (f.eks. med Docker).
9. **Dokumentation:** Sørg for at dokumentere hele processen, både med hensyn til arkitekturbeslutninger, API'er, og microservices. Dette inkluderer også dokumentation af den nye lagerstyringsservice og hvordan denne interagerer med resten af systemet. Dette kan gøres I jeres README fil.